

是缝合，而非一击：一种面向分解式大模型生成的、可机器校验的工作追踪纪律

Tony Gair · 2026 年 9 月 22 日 预印本。The Dark Factory Ltd, 英国南希尔兹 (South Shields)。

译文；以英文正本为准 (*Stitches, not shots: a machine-checked work-tracking discipline for decomposed LLM generation*)。术语与判断以英文版为最终依据。

摘要

要求语言模型“一击”(one shot) 产出一份从规格到证明的制品时，其失败方式很难被察觉：结果看起来言之成理，却没有任何记录能说明“应交付什么”与“实际交付什么”之间的差距，而“已完成”最终只是模型自己的断言。本文报告一种为消除这最后个性性质而构建的工作追踪纪律。一项应完成的工作单元在被着手之前，就先被记录为一个 **mūi**——一枚具名的缝合 (stitch)，携带一个有界的范围 (scope) 与一个确定性校验 (deterministic check)；而它只有在其校验通过、且该校验是在作者本人机器之外运行时，才被认定为完成，从而没有任何作者可以给自己的作业打分。状态从不作为标志位存储，而是通过运行校验重新计算得出，因此一个悄然不再为真的断言会自行退回松弛 (loose) 状态。我们描述这一原语、“先落针后动工”(lay-before-work) 规则、以及作者之外的认定闸门，并论证这种分解在错误定位、来源可溯与失败时的诚实性上何以胜过一击式生成。作为证据，我们报告一座正在运行的工厂，其决策部件都是经过证明的 SPARK 核，每一个都由散文规格锻造而来、并在作者之外被认定；以及一次以此类缝合为闸门的公开发布 (Crucible v0.2.0, 2026 年 9 月 22 日)。我们如实给出完成率——它们只是少数运行中才达成，而在一台陌生人机器上、以一个公开模型进行的洁净容器运行，七次里零次完成——并论证：贡献在于这套纪律，而非完成率。当系统说“已证明”时，是一个独立的校验器如此宣告；当它无法证明时，它会说明原因。

1. 引言

让一个能干的模型把散文规格转化为经过验证的软件部件，你常会得到一个能编译、读起来通顺、却是错误的东西。这种失败具有三个使其危险（而非仅仅令人失望）的性质。

- 它是无声的。一个比规格更弱的契约、一个什么也没确立的证明、后置条件里被丢掉的一个 **not** ——每一个都产出一个看起来已完成的制品。
- 它是无法归因的。一次一击运行不留下任何持久记录来说明“应交付什么”对“实际交付什么”，因此后来的读者无法把一处缺口与一次决定区分开。
- 除了模型本身，它无人能验证。完成与否，取决于生成者自称已经完成。

本能反应是去找一个更好的提示词。我们主张：解法不是更好的提示词，而是更好的记账单位。“智能体式”系统用来组织多步工作的那些机制——待办清单、

任务队列、先规划后执行的脚手架、草稿本式的计划——记录的是意图，不携带校验。一个被打了勾的条目仍然只是一个断言。所缺的，是这样一种“应完成工作”的单位：它从被写下的那一刻起，就携带着将裁定它是否完成的机器测试，并附带一条规则——该测试由声称完成的那一方之外的人来运行。

我们把这个单位称为 **mũi**。这个词是越南语中“缝一针”或“绣一针”的意思 (**mũi thêu**, 一枚绣花针脚; **mũi kim**, 针尖)，刻意选它而非“task”——在本文场景下 task 与目标语言的一个保留字冲突——并刻意选一个便于口头念出的词：人们可以隔着房间说“六枚已锚定，十一枚松弛，一枚已散脱”。¹一枚缝合所经历的状态则沿用英文针线活的词汇，因为在那里它们承载论证：一枚缝合在有东西固定它之前是松弛的 (**loose**)，一旦有证据把它固定住便是已锚定的 (**anchored**)，正如针线活中的锚定是把一根线固定好、使活计不会被扯开)，而当一个曾经成立的校验悄然不再成立时，它就散脱 (**fray**)。

贡献。

1. **mũi** 原语：标识符、范围、确定性校验，以及一个其状态被重新计算而非存储的两态生命周期 (§3)。
2. 先落针后动工纪律：计划中每一项应完成的工作都写成一枚 **mũi**，而非写成散文，从而使计划成为一块可校验的看板，而非一段叙事 (§3-4)。
3. 作者之外的认定闸门：校验在作者机器之外运行，作者永远不能自我认定；关闭一枚缝合的，是一份来自独立校验器的签名收据 (§4)。
4. 一份经验报告 (§6)：一座在此纪律下建成的、由经过证明的核组成的真实工厂，包括一次在陌生人机器上测得的公开发布，且完成率如实、不加粉饰地给出。

我们回答一个狭窄的问题——把应完成工作记录为带校验的缝合、并在作者之外认定，是否胜过一击式地产出整份制品？——答案是有保留的“是”，并对这条链中本纪律确实保护了哪一部分、未保护哪一部分保持精确。

2. 背景与相关工作

证明携带代码 (**Proof-carrying code**)。“生产者付出努力、消费者廉价地校验一份证书”这一思想源自 Necula 与 Lee [1, 2]。我们报告的纪律是它在管理层面的一个表亲：证明携带代码把证书附到已交付的制品上，而我们把校验附到一项应完成工作的单位上，并且仅当一个独立方运行该校验时才认定这项工作。二者可以复合——一份配套报告 [7] 把可独立校验的证书附到同一座工厂验证过程中的 SMT 步骤上——但这里的贡献是追踪纪律，而非证书格式。

智能体式任务追踪。先规划后执行以及反思式的智能体架构——先推理后行动的循环、先规划后求解的提示、在草稿本上的自我反思、对采样轨迹做自洽 [3, 4, 5, 6]——能很好地组织多步生成，但它们对进展的记录是一份意图清单。它们都没有把“将裁定某项工作是否完成的确定性测试”附到该应完成条目上，也都没有把“声称完成的一方”与“验证它的一方”分开。把一个条目标记为完成，仍然是生成者的一个断言——而这恰恰正是使一击式失败无声无息的那个性质。

¹这种混合词汇——越南语的单位名词配英文针线活的状态词——是作者刻意的命名选择，意在尊重这一隐喻所借用的手艺，而非把一个英文词硬塞进这门手艺。此处的释义为作者所拟，未经母语者校订；在将该术语用于翻译材料并倚重它之前，此项校订仍待完成。

测试驱动开发（TDD）在写代码之前先写测试，是“先落针后动工”最接近的日常类比。差别在组织层面：TDD 的测试由写代码的同一个开发者、在同一台机器上运行，通过与否是自我报告的。müi 纪律加上了作者之外的认定规则——校验是信任的单位，且由不是作者的人来运行。

面向智能体的结构化、概率化决策。与本文同期的一条路线让智能体的决策变得结构化，却仍然保持概率化。Jev 是 TypeSafe AI 的一个“System-One”（系统一）分类器 [9]，给它一个状态与若干类型化问题，它返回类型化的答案并附带概率分数，据称在分类任务上“推理速度最高快 200 倍、成本最低降至 1/400”（相较于一个可比的模型）；其所述用途是模型路由、以及在一次工具调用运行前对其做评估的安全护栏。它是 müi 的一个富有启发性的反面，而这一对照使论旨更加锋利。一个 Jev 答案是一个带置信度的快速启发式裁决——系统一；一枚 müi 的认定是一个可重跑的校验——系统二。这一差别在其承重之处最为要紧。一道概率化护栏依据一个分数来裁决，而一个分数可以被它正在评判的那个输入本身所移动；一个能够被它所控制之物饿死或引偏的控制，在极限意义上并不是一个控制。一枚 müi 的校验在每次被读到时重跑，无法被说动而改变其裁决——这正是“重新计算状态而非存储状态” (§3) 的全部要义。二者可以干净地复合，我们也不作相反主张：一个系统一分类器是廉价初筛路由器的合适工具，而一个经过证明的核是一项必须成立的决定的合适权威。论旨所坚持的仅仅是那条边界——一项承重的决定，即另一个部件或一个人将会倚赖的决定，最终不得搁在一个概率之上，无论那个概率获取起来何等廉价而快速。（对于这样一个证明，“必须成立”能主张什么、又不能主张什么，见 §7。）

SPARK 与 gnatprove 作为判定神谕（oracle）。SPARK 是 Ada 的一个可验证子集；gnatprove 用 SMT 后端消解验证条件，并返回一个确定性裁决。它给了我们一个真正的判定神谕，而非一个启发式：对于一个由散文锻造的核，“证明器关闭了每一项义务”是一个第二台机器可以重新推导的事实。本纪律所强制的那种分离——要证明，而不仅仅是生成——依赖于在该领域拥有这样一个判定神谕。

AI 产出的经验证软件。与本文同期的工作报告了由 AI 智能体大规模产出的经验证安全软件 [8]。那项工作处理的是制品的生产；这里的问题是正交且更上游的——在这类制品被生产的过程中，应完成的工作如何被记录与认定，从而使“已完成”永远不是生成者的一面之词。

3. müi 原语

一枚 müi 由四样东西构成：

字段	是什么
id	为这项应完成的断言取的一个稳定、可念出的名字（例如 door-is-sole）
范围（scope）	这项工作被允许触碰的路径或制品的集合
校验（check）	一条 shell 命令，当且仅当该断言成立时退出码为 0

字段	是什么
状态 (state)	派生而非存储：松弛或已锚定（另有散脱与掉针，用于下文的失败与链式情形）

有两条性质是承重的。

断言写成“完成时将为真的东西”，而非“要做的工作”。“该更正已被提交”——绝不写成“提交该更正”。工作可以在断言仍为假时被执行（一次并未真正应用修复的提交），因此校验测试的是断言，而非活动本身。如果你写不出校验，你就没有一枚缝合；你只有一个意图，而工具会如实地把它记为意图——每次被读到时都如此——而不让它伪装成被追踪的工作。

状态通过运行校验重新计算，从不从标志位读取。刻意地没有“标记为完成”。向看板询问状态，就会运行每一个校验并报告哪些仍然松弛。这就是击败无声陈旧化的那一条性质：一个上周被锚定、其校验今天却失败的断言，会自行退回松弛，无需任何人去动它。一份散文式的应做清单是一个存储的标志位——写下时为真，之后从不重读；而一块由 mūi 组成的看板，不给任何东西留下自己批改自己作业的余地。

先落针后动工。标准规则是：计划或工作笔记中每一项应完成的条目，都在你本会写下“仍欠”的那一刻被落成一枚 mūi。计划由此成为一块看板，而非一个故事。读看板回答的是对一份计划唯一值得问的问题——不是“状态如何”，而是“还欠什么”——而这个答案是被计算出来的，不是被回忆出来的。

链 (chains)。一枚 mūi 可以穿过另一枚被织出，如同链式针脚：只有当它上游的一切都被锚定时，它才能被锚定。一枚自身校验通过、却坐落在一枚失败的上游缝合之上的缝合，是掉针 (dropped)——这是“修复属于上游、不在此处”的信号。落针时会拒绝成环。链使得一道发布闸门——“该制品适合交付”——可见地坐落在它所依赖的那些具体断言之上，从而一次上游回归会把整条链拉回松弛，而不是在顶端留下一盏陈旧的绿灯。

范围内敛 (scope containment)。在一枚缝合下被认定的工作，只能触碰该缝合所声明范围以内的路径。一枚为改动某个发射器而落的缝合，无法携带对前门的改动。这正是使看板能够做出“此断言成立”之外的第二重承诺的东西：在这次修好的过程中，邻居可被证明地未被触碰。

4. 纪律：生命周期与作者之外的认定

生命周期刻意地很短：落针 → 在范围内动工 → 运行校验 → 仅当通过时锚定。一个失败的校验不会大声报错并停下；它只是让缝合停留在松弛而诚实的状态。从“我做了这项工作”到“它完成了”，没有一条不经过校验的路径。

把这一点与自我追踪的 TDD 区分开来的性质，是作者之外的认定。

- 校验在作者机器之外运行。对于一个被锻造的核，重新证明它的机器与产出它的机器是分开的，并在那里签发一份收据。作者的一面之词不是可采纳的证据；收据才是。

- 作者永远不能自我认定。认定闸门由一个经过证明的部件裁决，而非由动工的那一方裁决；宣布一枚缝合”已批准”的权力完全掌握在作者工具集之外。这迫使一个被明确对待的设计问题：若无一个外部批准者，一道认定”由缝合携带的工作”的闸门，只要落一枚缝合即可被绕过——即自我授权。因此闸门被建成这样：落一枚缝合不授予任何许可；一个独立的、由人掌握的动作才批准它，而且该改动还必须落在该缝合的范围之内。
- 改动携带缝合的 id。每一次被认定的改动都点名它所了结的那枚缝合，因此记录显示的是哪个断言携带了哪次改动，认定日志读起来是一本”被兑现的断言”的账，而非一本”被触碰的文件”的账。

我们对这道闸门的强度保持精确。在当前系统中，重新证明的机器与锻造的机器是分开的、并签发自己的收据，但锻造方 仍然在证明器上持有管理员权限。在这一点被移除之前，监管链是测得的，而非被担保的——“测得的”是对它诚实的用词。一种已被演示的失败方式使这一点更加尖锐：一份点名某个独立证明器的收据，可以仅凭环境变量、无需任何特权 访问就被伪造出来，用于一个从未被任何人真正证明过的部件。因此，除非证明器的独立性本身被强制，收据格式就是 生产者自我断言的；本纪律把这陈述为一项局限，而非加以掩饰。

5. 它何以胜过一击式生成

论证不是缝合使模型变得更好。而是它们改变了必须被信任的东西。

- 错误定位。一份失败的一击式制品交出的是单一的失败整体。一条失败的缝合链交出的是一枚具名的失败缝合，其邻居可被证明地未被触碰，从而缺陷被定位到一个范围，而非被抹开在整份制品之上。
- 信任的单位是校验，不是模型。一枚”看起来完成了”但其校验失败的缝合，停留在松弛。再流畅、再言之成理的 输出也无法移动它。这直接回应了无声失败的那个性质：言之成理不可采纳；校验才可采纳。
- 单调的诚实。因为状态被重新计算且没有”标记为完成”，记录只能朝真相移动。一枚缝合无法靠断言被了结，而一枚不再成立的缝合会在下一次被读到时自行退回。
- 来源可溯。每一次被认定的改动都点名它的缝合、并携带一份签名的、作者之外的收据，因此”应交付什么对实际 交付什么”是一次对看板的查询，而非一次对记忆的重建。
- 有界的波及范围。范围内敛意味着一枚坏的缝合无法悄然改动一个经过证明的邻居——正是这一性质使得在一大批 已被证明的工作之上继续建造是安全的。

性质	一击式生成	mūi 纪律
失败的定位	整份制品	具名缝合，带范围
被信任的是什么	模型的输出	校验 + 作者之外的证明器
“已完成”的来源	模型的断言	点名校验器的签名收据
失败时的诚实	看似通过	停留松弛；散脱在被读时退回
可复现性	重跑整个生成	在任意机器上重跑校验

一击式拥有、而本纪律舍弃的那一个性质，是抵达一个言之成理答案的速度。

而这恰恰正是使一击式失败危险的那个 性质。

6. 经验报告

这座工厂。所讨论的系统由散文规格建造 Ada/SPARK 部件，并拒绝交付任何它无法证明的东西。它本身就是按它 建造东西的方式被建造的：做出其决策的部件都是经过证明的 SPARK 包，且每一个都由工厂从散文产出、而非手写。它们中的每一个都是经由上述纪律进入服役的——落成一枚缝合、在范围内锻造、在作者之外重新证明、仅凭一份签名收据被 认定。

案例研究——Crucible v0.2.0，2026 年 9 月 22 日公开。该发布²以一条缝合链为闸门——前门是唯一入口、交付的制品不依赖任何缺失的部件、交换路径不泄露作者的任何东西——其决策部件都是经过证明的、由缝合了结的核。我们如实给出完成数字，因为本纪律的价值并不取决于它们好看。

所测量的对象	结果
仅”分解”阶段（两张规格单）	修好一处提示后 50 之 50；修好之前 32 之 50
整条流水线，散文 → 经证明的单元（首次 A/B，在 40 次中的第 19 次提前停止）	3 之 10，与 3 之 9
整条流水线，某一”计数”规格	2 之 8——且这两次中有一次比其规格更弱
整条流水线，后来对某候选改进的 30 次 A/B	33% 对 43%；不具结论性，未被采用
陌生人机器上的洁净容器、公开模型、未拷贝我们的任何东西	0 之 7

整条流水线只在少数运行中完成；主要的卡点是契约发射——模型在写出一个证明器能接受的契约之前就耗尽了它的 轮次预算。在一台陌生人的机器上、在一个洁净容器里、以一个公开模型且没有我们的任何调优，它在七次尝试里零次完成。我们报告这些而非略去它们，因为它们是这一主张的诚实框架，而这个主张更狭窄、我们认为也更有用：当系统说已证明 时，是一个独立的证明器如此宣告；当它无法证明时，它点名它停在哪个阶段，并保留那个案例而非丢弃它。

最能印证这套纪律的发现，是它在自身内部逮住的一次失败。两天之内两次，模型发射了一个比规格所要求的更弱的 契约。其中一个通过了当时所有的闸门，包括那个针对”什么也没说的契约”的校验；另一个丢掉了一个 not，那本会让一个案例永远悬而未决，而它仅仅是碰运气地经由一个无关的语法错误才被逮住。这就是本纪律自身的局限被具体化了 (§7)：一枚缝合的诚实程度只等同于它的校验，而一个测试”主体满足契约”的校验，并不测试”这契约就是你所意指的那一个”。本纪律之所以让这道缺口浮现，恰恰是因为它强制校验被点名并被审视；一次一击式运行本会把那个言之成理的 制品直接交付出去。

²github.com/the-dark-factory/Crucible_AGPL，标签 v0.2.0。

7. 局限，以及何时一枚 *mūi* 是错误的工具

我们把这些如实陈述：这套纪律很容易被过度解读。

1. 一枚缝合的诚实程度只等同于它的校验。整套方法把信任归约为校验的质量。一个能通过“证明了个空”的校验会给出虚假的安慰。§6 里那些比规格更弱的契约就是这一失败的现实版本：“已证明”意味着主体满足了系统所写的契约，而非作者所意指的契约。一个把契约强度与某个基准真值相比较的校验，已被设计但尚未建成；在它建成之前，必须由人来阅读所发射的契约。
2. 作者之外的闸门是测得的，而非被担保的。如 §4 所述，面对一个持有管理员权限的生产者，证明器的独立性尚未被强制，而一份看起来有效的收据可以在没有真实证明的情况下被伪造。这是这一信任主张的承重开放问题。
3. 有些工作抗拒分解，有些校验本身不可判定或代价高昂。当一项应完成的断言没有一个廉价的确定性测试时——例如任何取决于精确浮点行为的东西，在此处于入口就被拒绝而非在证明处失败——一枚缝合就是错误的工具，而诚实之举是把这一点说出来，而不是去落一枚其校验无法回答的缝合（看板会把它报告为散脱）。
4. 一个好校验的成本是真实的。写校验往往和做这项任务本身一样费力，而本纪律把这份成本前置。它的回报在于失败时的诚实与来源可溯，而非抵达第一个答案的速度。
5. 证据是小样本、且部分是自我测量的。完成率来自少数几张规格单；陌生人机器那个数字是一次单独的洁净容器运行；而监管链是测得的、而非被担保的。我们主张的是这套纪律，而非这些数字的普遍性。
6. 此处的一个证明不是资质证据 (**qualification evidence**)。这些证明是真实且可重跑的，但产出它们的工具链，并不是一条经过认证的安全攸关计划所要求意义上的合格 (*qualified*) 工具链 (DO-178C、EN 50128、ISO 26262 各自都建立在一个合格工具之上，附带其自身的制品、可追溯性，以及一个承担责任的供应商)。因此，一枚缝合的收据是读者可以重新推导的东西；它不是一个可以作为资质证据出示的东西。这样一个核可以充当一条认证计划的输入，但不能替代其资质认证。当 §5 说一项决定“必须成立”时，这就是那个主张所能触及的上限。

8. 结论

一枚带校验的缝合，是比一项带断言的任务更好的记账单位。把应完成的工作记录为一枚具名、带范围、被确定性校验的缝合——在动工之前落下、其状态在每次被读到时重新计算、且仅在作者本人机器之外被认定——就把一个无法验证的一击式生成问题，转化为一串可被独立校验的问题。它并不使模型更频繁地完成；按我们的数字，它只在少数时候完成。它使之可靠的，是“已完成”的含义：永远不是生成者的断言，永远是一个第二台机器能重跑的校验。正是这一点，让一座由经过证明的核组成的工厂得以由一个语言模型建成、并被发布，而这次发布的任何部分都不倚赖模型的一面之词。

制品与可复现性

§6 所引用的公开发布随附源代码、一份证明摘要，以及一个在读者机器上重新校验每一个核的脚本。§6 中的完成数字是 作者自己在固定版本下的测量；洁净容器那个数字，任何读者都可以用一个公开模型、从该公开发布出发予以复现。

参考文献

- [1] G. C. Necula. *Proof-Carrying Code*. POPL '97, pp. 106 – 119. DOI: 10.1145/263699.263712.
- [2] G. C. Necula and P. Lee. *Safe Kernel Extensions Without Run-Time Checking*. OSDI '96.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.
- [4] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee and E.-P. Lim. *Plan-and-Solve Prompting*. ACL 2023.
- [5] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan and S. Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023.
- [6] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery and D. Zhou. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR 2023.
- [7] T. Gair. *Proof-Carrying Verification Conditions for SPARK: An Experience Report*. 预印本, The Dark Factory Ltd, 2026 年 8 月。DOI: 10.5281/zenodo.22902004.
- [8] J. Philipp. *The Prover Is the Judge: Verified Security Software from AI Coding Agents in Ada/SPARK*. arXiv:2607.14340, 2026 年 7 月。
- [9] TypeSafe AI. *Jev: a System-One model for structured decisions / Building a harness with Jev*. langchain.com/blog/building-a-harness-with-jev, 2026。产品公告；其速度与成本数字（“推理速度最高快 200 倍、成本最低降至 1/400”）为厂商在分类任务上的自述主张，按其自述引用。

书目说明。条目 [1]、[2]、[7] 与 [9] 已对照第一手来源核实（[9] 对照厂商公告；其性能数字为厂商主张，本文未 独立测量）。条目 [3]–[6] 与 [8] 在超出预印本存放的任何投稿之前，应对照出版方记录（会议/期刊、作者名单、页码）予以确认。

工具与 AI 使用声明

本工作的开展、以及本报告的起草，是在一次借助 Anthropic 的 Claude（Claude Code）的 AI 辅助工作会话中完成的。作者是神经多样性者并远程工作；AI 辅助被用作一种无障碍便利，正如另一位研究者可能与一位誊写员协作那样。

依据通行的会议/期刊政策，AI 系统不是本工作的作者，也不作为作者被署名。作者对其内容负全部责任。§6 中的测量由 具名工具产出，且无论产出它的会话

如何进行，都可从公开发布予以复现；§4 与 §7 中的判断——本纪律未保护什么、信任究竟停留在何处——已尽作者所能狭窄地陈述，恰恰因为 AI 辅助会使夸大变得流畅而容易，而这些判断出自作者本人。